

BPFOPTBENCH: A Benchmark for Agentic Optimization of Existing eBPF Programs

Anonymous Author(s)

Abstract

Existing eBPF programs are attractive optimization targets for agents because the action space is constrained, feedback is executable, and good decisions require systems judgment. They are also difficult to evaluate. A candidate rewrite must be accepted by the Linux verifier, preserve the application loader and workload behavior, and improve the final JITed code despite noisy runtime measurements. We propose BPFOPTBENCH, a benchmark for agents that tune existing eBPF programs under real verifier, JIT, and workload feedback. BPFOPTBENCH defines frozen optimization tasks, constrained action spaces, raw and structured feedback, hidden evaluator checks, and component metrics for validity, workload success, performance, regret, cost, and integrity failures. Historical corpus data motivates the benchmark because no-op floors are large, rewrite counts do not predict speedups, and tuned human policies improve some cases without solving the corpus. These results argue for a benchmark and oracle first, before claiming that current agents optimize eBPF well.

Keywords: eBPF, agents, optimization, benchmark, verifier, JIT

1 Background

eBPF lets applications extend the Linux kernel with small programs that are loaded through application-specific control paths, checked by the in-kernel verifier, and lowered by an architecture-specific JIT before running on events such as packets, system calls, and profiling samples [2, 4]. Prior work shows that eBPF has real optimization and generation headroom. K2 optimizes BPF bytecode before load [7], Merlin combines LLVM and bytecode transformations [5], and LLM-based eBPF systems explore code generation and assistance [1, 9]. Agent benchmarks such as SWE-bench, τ -bench, and KernelBench show how models can be evaluated in executable environments with hidden tests, interaction budgets, or performance oracles [3, 6, 8]. BPFOPTBENCH combines these threads, but changes the unit of evaluation. The benchmark does not ask an agent to write a fresh program or pass a verifier-only repair task. It asks whether an agent can choose a useful optimization action for an existing eBPF application under the real verifier/JIT/workload loop.

2 Motivation

Our current data supports a benchmark claim rather than an agent-performance claim. Table 1 summarizes the strongest evidence. The core lesson is that verifier acceptance, rewrite

applicability, JIT size, workload success, and performance counters do not agree automatically. A benchmark that scores only local rewrites or accepted bytecode would reward noise chasing and hide regressions.

Table 1. Existing data motivates BPFOPTBENCH as a benchmark. Ratios are post/base unless noted.

Evidence	Result	Benchmark implication
No-op floors	no-op ReJIT 0.9019 and skip-ReJIT 0.8587 over 147 programs	agents need noise gates and no-op controls
Pass audit	no completed standalone pass has a paper-ready improvement	rewrite count is not enough evidence
Full corpus	146 programs, geomean 1.0038, 81/65 W/L	headline performance can be nearly flat
Applied split	changed-code rows 1.0323, unchanged rows 0.9827	event mix and phase effects can dominate
Human policy	shared subset improves 0.891x to 0.898x in speedup-style reporting	choosing not to optimize is a real policy decision

3 Design

BPFOPTBENCH adds a benchmark layer above an existing eBPF runner. The runner executes real applications and records raw counters. BPFOPTBENCH defines the task, permitted actions, feedback surface, hidden evaluator, and scoring protocol. Each task names a subject application, workload, architecture, baseline artifact, action space, attempt budget, protected files, and public and hidden checks. The first track focuses on actions that the current infrastructure can exercise, including suite-wide pass lists, per-app policies, and per-program policies over bytecode and live ReJIT actions. The kinsn-backed JIT capabilities are an optional backend, not the definition of the benchmark. Source-level and LLVM-backend actions can be added later as adapters that rebuild applications and feed the same oracle.

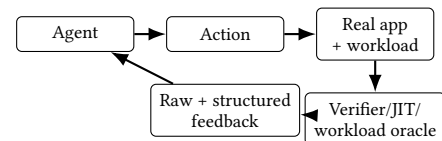


Figure 1. BPFOPTBENCH evaluates a closed loop in which an agent observes, chooses an allowed optimization action, executes the real app path, and scores with a composed oracle.

The oracle has hard gates before any performance score is assigned. A candidate must be accepted by the real verifier path, preserve the application lifecycle, pass workload-specific correctness checks, clear the task’s performance threshold or measured noise gate, and avoid forbidden shortcuts such as changing the workload, lowering the run budget, replacing app loaders, filtering ReJIT failures, or fabricating result files. The executor records raw artifacts only. Ratios, geomeans, win/loss counts, confidence intervals, and scoreboards are computed by post-hoc analysis scripts so interpretation never hides inside the measurement path.

4 Preliminary Evaluation

We structure the evaluation as a benchmark-artifact study first and an agent-performance study second. The latest v3 raw run already shows that the execution substrate can complete the six supported macro apps (bcc/set, tracee/monitor, tetragon/observer, ktrn, cilium/agent, and otelcol-ebpf-profiler/profiling) through the native-loader path and record per-app raw payloads. Because that run enabled no optimization passes, it is substrate evidence rather than optimization evidence.

Table 2. Preliminary evaluation plan and the claim each block can support.

Block	Method	Claim supported
Difficulty study	reuse no-op floors, pass audit, corpus split, and policy comparison	eBPF optimization needs a composed oracle
Task/evaluator audit	freeze 20–50 tasks with protected workloads and hidden checks	BPFOPTBENCH is a reproducible benchmark artifact
Offline agent study	ask agents to judge signals, choose next experiments, and detect invalid actions	feedback effects can be measured
Small live scoreboard	compare no-op, static, random/grid, human policy, and agents on A1–A3 tasks	policies can be compared under equal budgets

The first paper should report component metrics rather than one opaque score, including validity rate, workload success rate, performance success rate, bpfopt_success_p, regret to the best-known valid action, attempts, wall-clock cost, token cost, and integrity failure rate. With 20–50 audited tasks, BPFOPTBENCH can support a workshop benchmark claim. A stronger standalone benchmark would require a larger public development split, a private or rotating hold-out split, and fresh-VM replay checks. The immediate goal is therefore calibrated. We should freeze tasks and the hidden evaluator, then run a small scoreboard that compares no-op, static, random, human-tuned, and LLM-agent policies without claiming that current agents already solve eBPF optimization.

References

- [1] Xiangyu Gao, Xiangfeng Zhu, Bhavana Vannarth Shobhana, Yiwei Yang, Arvind Krishnamurthy, and Ratul Mahajan. 2025. Offloading

- the Tedious Task of Writing eBPF Programs. In *Proceedings of the 3rd Workshop on eBPF and Kernel Extensions* (Coimbra, Portugal) (eBPF ’25). Association for Computing Machinery, New York, NY, USA, 63–69. <https://doi.org/10.1145/3748355.3748369>
- [2] Bolaji Gbadamosi, Luigi Leonardi, Tobias Pulls, Toke Høiland-Jørgensen, Simone Ferlin-Reiter, Simo Sorce, and Anna Brunström. 2024. The eBPF Runtime in the Linux Kernel. arXiv:2410.00026 [cs.OS] <https://arxiv.org/abs/2410.00026>
- [3] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2023. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? arXiv preprint arXiv:2310.06770. <https://arxiv.org/abs/2310.06770>
- [4] Linux kernel documentation. 2025. eBPF verifier. <https://docs.kernel.org/bpf/verifier.html>. The Linux Kernel documentation, Documentation/bpf/verifier.rst.
- [5] Jinsong Mao, Hailun Ding, Juan Zhai, and Shiqing Ma. 2024. Merlin: Multi-tier Optimization of eBPF Code for Performance and Compactness. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (La Jolla, CA, USA) (ASPLOS ’24). Association for Computing Machinery, New York, NY, USA, 639–653. <https://doi.org/10.1145/3620666.3651387>
- [6] Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. 2025. KernelBench: Can LLMs Write Efficient GPU Kernels? arXiv preprint arXiv:2502.10517. <https://arxiv.org/abs/2502.10517>
- [7] Qiongwen Xu, Michael D. Wong, Tanvi Wagle, Srinivas Narayana, and Anirudh Sivaraman. 2021. Synthesizing safe and efficient kernel extensions for packet processing. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference* (Virtual Event, USA) (SIGCOMM ’21). Association for Computing Machinery, New York, NY, USA, 50–64. <https://doi.org/10.1145/3452296.3472929>
- [8] Shunyu Yao et al. 2024. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv preprint arXiv:2406.12045. <https://arxiv.org/abs/2406.12045>
- [9] Yusheng Zheng, Yiwei Yang, Maolin Chen, and Andrew Quinn. 2024. Kgent: Kernel Extensions Large Language Model Agent. In *Proceedings of the ACM SIGCOMM 2024 Workshop on EBPF and Kernel Extensions* (Sydney, NSW, Australia) (eBPF ’24). Association for Computing Machinery, New York, NY, USA, 30–36. <https://doi.org/10.1145/3672197.3673434>